

Aave Labs - Stable Vaults - Extension 1

Final report

Prepared by: Josselin Feist (josselin@seceureka.com)

July 6th, 2026

Prepared For: Aave Labs

Executive Summary	5
Overview	5
Appendix A: Ad-hoc recommendations	6
Access controls	6

Executive Summary

Overview

During the week of February 16, we performed a one-week review of the Stable Vault contracts, covering the integration of price and balance Oracle feeds (PR157) and the access control integration (PR158).

We reported some mismatches in the access control schema (e.g. `replayFundsReceiving` was missing) which are not included as standalone findings in the report as they were fixed during the week.

Moreover, additional ad-hoc recommendations shared during the extension are summarized in [Appendix A](#).

The addition of the Oracle improves the system by simplifying the bridging protocol and reducing the code complexity. However, while the Oracle partially prevents the risk of a depeg (e.g. by preventing deposits), it does not protect assets that have already been deposited or donated (which are inherent to the protocol allowing “1:1 swap across chains”).

We noted that the new access controls (PR 158) provide a fine-grained control over each function. Further improvements could be made, such as filtering functions per address to reduce the risk of collisions or similar function names. However, given the current complexity, we recommended not increasing the granularity further unless a specific need arises. See [Appendix A](#).

Finally, we recommended Aave Labs to prepare an incident response plan, in particular to be prepared in case assets depeg or similar risks.

This report is part of a multiple stages review of the Stable Vault (2/4).

Appendix A: Ad-hoc recommendations

The following are ad-hoc recommendations made during the 1 week extension (February 16th):

- **Change the destination of the asset transfer in `claimSurplusInterest`**, to isolate the risk from the caller
 - Implemented in PR 171
- **Use a loop for role assignments in `RolesLib`**. The repetitive code pattern can be simplified through a loop
 - Implemented in commit a32a05b383f937f507365b4286fc7ca43cccb76b
- **Carefully monitor solc versions after 0.8.34**. A compiler issue was discovered and fixed on transient usage (https://x.com/solidity_lang/status/2024181650155065848). While the issue does not impact Aave's usage of transient, it does raise concern of potential related bugs. We recommend that Aave Labs
 - Keep 0.8.28
 - Use 0.8.34, but wait a 2 weeks period before deploying (to avoid any residual bugs to be found in the compiler)
 - Drop the usage of transient in the reentrancy modifier
- **Validate the following assumptions:**
 - The Rebalancer can pause any deposit by calling `disableDepositsToStrategy`
 - Assets cannot be delisted from `AssetRegistry`
 - *Aave Labs mentioned that these behaviors are expected*

Access controls

The new access controls provide a fine-grained control over each function by assigning each function to a role id. This has the following benefits:

- Allowing a per-function control
- Merging together functions that are shared across contracts with the same access (ex: `rescue native/token`, as in Figure 1)

For the current architecture, the schema is sufficient. However, as discussed with Aave Labs, this system would be limited if:

- Function ID collision between contracts happens
- The same function is shared across contracts but requires different callers
- A per-contract setup would be needed (ex: for `claimMerkleRewards`)

BaseChainGateway	rescueTokens(address,uint56)	Main Admin	Secondary Admin	Disabler
BaseChainGateway	rescueNative(uint256)	Main Admin	Secondary Admin	Disabler
FundsHandler	rescueTokens(address,uint56)	Main Admin	Secondary Admin	Disabler
FundsHandler	rescueNative(uint256)	Main Admin	Secondary Admin	Disabler
CcipAdapter	rescueTokens(address,uint56)	Main Admin	Secondary Admin	Disabler
CcipAdapter	rescueNative(uint256)	Main Admin	Secondary Admin	Disabler
StableVault	rescueTokens(address,uint56)	Main Admin	Secondary Admin	Disabler
StableVault	rescueNative(uint256)	Main Admin	Secondary Admin	Disabler

Figure 1: Access controls spreadsheet

A more granular control would require adding either the contract's name or address to the role ID, to uniquely separate each function ID per contract. Given the number of existing roles (40+), such an addition would create substantial complexity within the deployment script, for no direct benefit. We therefore recommend not following this pattern at this stage.

If further granularity is required in the future, it might be beneficial to separate the access controls in different categories:

- Per function
- Per function + contract name (optional)
- Per function + contract address

And use the appropriate category when needed. Combining categories will increase the complexity, but will allow merging together similar access controls (ex: rescue native/token to the disabler), while allowing fine-grained access when needed (ex: having a separate caller to `claimMerk1Rewards` per vault deployed).

However we recommend to aim to avoid such additional complexity, and keep the access controls design as it is, as long as it fits the need.